

Chapitre 4: Estimateur des k plus proches voisins et estimateur de noyau

Contents

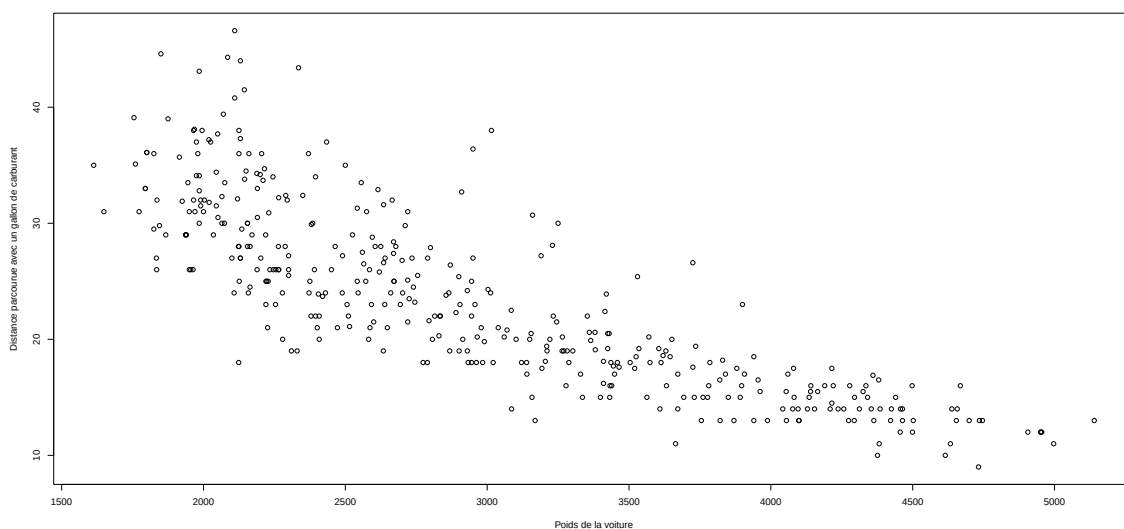
1	Estimateur des k plus proches voisins	3
2	Estimateur de noyau	8
3	Régression logistique et classification	13

Considérons le jeu de données **Auto** de la library **ISLR** du logiciel R. Ce jeu de données contient 392 observations. La variable réponse est **mpg**, la distance moyenne en miles parcourue par la voiture avec un gallon de carburant. On s'intéresse à une seule variable explicative, soit le poids de la voiture **weight**.

On commence par tracer le nuage de points $\{(X_i, Y_i), i = 1, \dots, 392\}$, où X_i et Y_i sont, respectivement, le poids et la distance moyenne en miles parcourue avec un gallon de carburant de la voiture i .

```
> if (!require("ISLR")) install.packages("ISLR")
> library("ISLR")#### Installer la librairie ISLR
>
> data(Auto)
> X <- Auto$weight
> Y <- Auto$mpg
>
> plot(X,Y,xlab="Poids de la voiture",
+      ylab="Distance parcourue avec un gallon de carburant")
```

Et on obtient



Ce graphique montre que la relation entre X et Y n'est clairement pas ~~clairement pas~~ linéaire. Dans ce chapitre, on considérera ~~plus~~ **plusieurs** approches pour étudier la relation entre X et Y dans cette situation.

1 Estimateur des k plus proches voisins

Supposons que la relation entre X et Y s'écrit sous la forme $Y_i = f(X_i) + \epsilon_i$, où f est une fonction inconnue à estimer et ϵ est un terme d'erreur qui vérifie $E[\epsilon_i|X_i] = 0$. On suppose que la fonction f est assez lisse. Autrement dit, on suppose que f est deux fois dérivable. Avec ces hypothèses, on a

$$f(x) = E[Y|X = x].$$

On suppose en outre que la variable X est continue. Autrement dit, il n'y a pas d'égalité entre les X_i ; i.e. $P(X_i = X_{i'}) = 0$ si $i \neq i'$.

Sous ces hypothèses, l'estimateur des k plus proches voisins estime la fonction f par

$$\hat{f}(x) = \frac{1}{k} \sum_{i \in N_k(x)} Y_i,$$

où $N_k(x)$ est l'ensemble des indices des k plus proches voisins de x . Pour définir rigoureusement $N_k(x)$, on pose $d_i(x) = (X_i - x)^2$ pour $i = 1, \dots, n$. On ordonne les distances $d_i(x)$ dans l'ordre croissant

$$d_{(1)}(x) < d_{(2)}(x) < \dots < d_{(n)}(x).$$

$N_k(x)$ est donc l'ensemble des indices i qui vérifient $\{d_{(i)}(x) \leq d_{(k)}(x)\}$.

Avec l'exemple précédent, on calcule $\hat{f}$ en utilisant différentes valeurs de k .

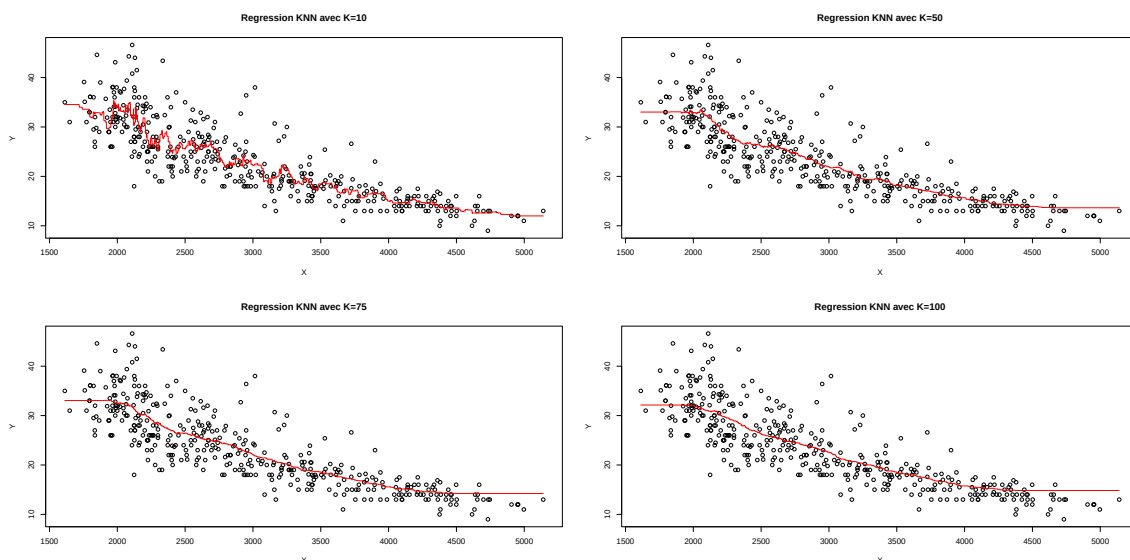
```
> if (!require("FNN")) install.packages("FNN")
Le chargement a nécessité le package : FNN
Message d'avis :
le package 'FNN' a été compilé avec la version R 4.4.2
> library(FNN)
>
> axe.x <- data.frame(lstat=min(X):max(X))
>
> pred_10 <- knn.reg(train=X,test=axe.x,y=Y,k=10)
> pred_50 <- knn.reg(train=X,test=axe.x,y=Y,k=50)
> pred_75 <- knn.reg(train=X,test=axe.x,y=Y,k=75)
> pred_100 <- knn.reg(train=X,test=axe.x,y=Y,k=100)
>
> par(mfrow = c(2,2))
>
> plot(X,Y,type="p")
```

```

> lines(axe.x$stat,pred_10$pred,col="red")
> title("Regression KNN avec K=10")
>
> plot(X,Y,type="p")
> lines(axe.x$stat,pred_50$pred,col="red")
> title("Regression KNN avec K=50")
>
> plot(X,Y,type="p")
> lines(axe.x$stat,pred_75$pred,col="red")
> title("Regression KNN avec K=75")
>
> plot(X,Y,type="p")
> lines(axe.x$stat,pred_100$pred,col="red")
> title("Regression KNN avec K=100")

```

Et on obtient



Ces graphiques montrent que le comportement général de $\hat{f}$ dépend du choix de k .

Examinons $\hat{f}$ de plus près. Lorsque X_i est proche de x , on peut écrire

$$E(Y_i) = f(X_i) \approx f(x) + f'(x)(X_i - x) + \frac{1}{2}f''(x)(X_i - x)^2$$

et donc

$$E[\hat{f}(x)] \approx f(x) + f'(x) \frac{1}{k} \sum_{i \in N_k(x)} (E[X_i] - x) + \frac{1}{2k} f''(x) \sum_{i \in N_k(x)} (E[X_i] - x)^2.$$

Si les X_i s sont symétriquement distribués autour de x , alors

$$\sum_{i \in N_k(x)} (E[X_i] - x) \approx 0$$

et

$$E[\hat{f}(x)] \approx f(x) + \frac{1}{2k} f''(x) \sum_{i \in N_k(x)} (E[X_i] - x)^2.$$

On montre que le biais $\sum_{i \in N_k(x)} (E[X_i] - x)^2 / (2k)$ croît avec k .

D'autre part, si on fait l'hypothèse que $Var[\epsilon_i] = \sigma^2$, alors on a $Var[\hat{f}(x)] = \sigma^2/k$. Ce terme décroît avec k .

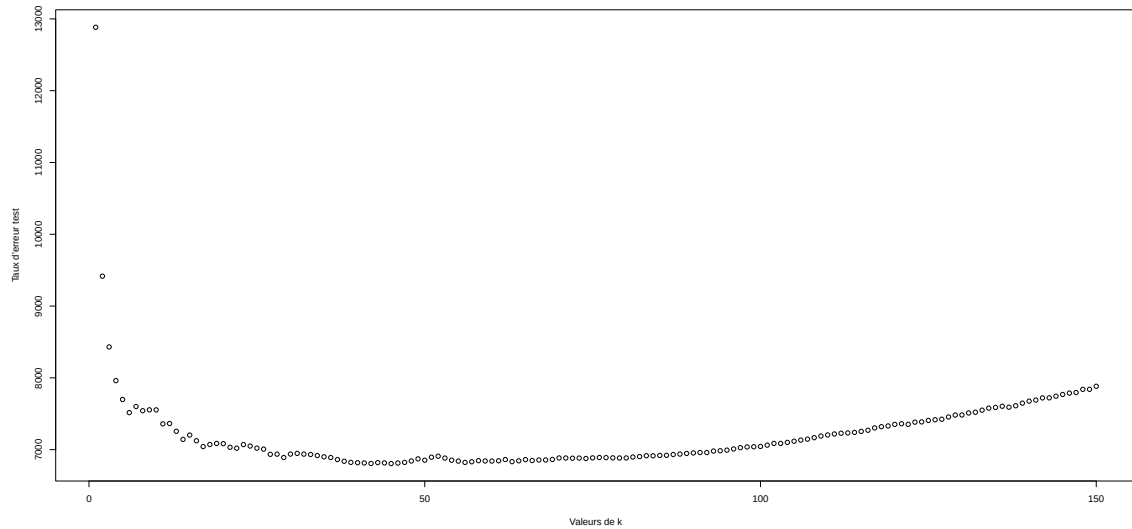
Il faut donc sélectionner k qui minimise l'erreur quadratique moyenne

$$biais^2[\hat{f}(x)] + Var[\hat{f}(x)].$$

Dans la pratique, on sélectionne k par la méthode de validation croisée.

```
> #####
> ## Selection de K ##
> #####
>
> valeurs.k <- seq(1,150)
> cv.k <- NULL
> for(k in valeurs.k)
+ {
+   reg.knn.cv <- knn.reg(train=X,y=Y,k=k)
+   cv.k <- c(cv.k,reg.knn.cv$PRESS)
+ }
>
> k.min <- valeurs.k[which.min(cv.k)]; k.min
[1] 45
>
> plot(valeurs.k,cv.k,xlab="Valeurs de k",ylab="Taux d'erreur test")
```

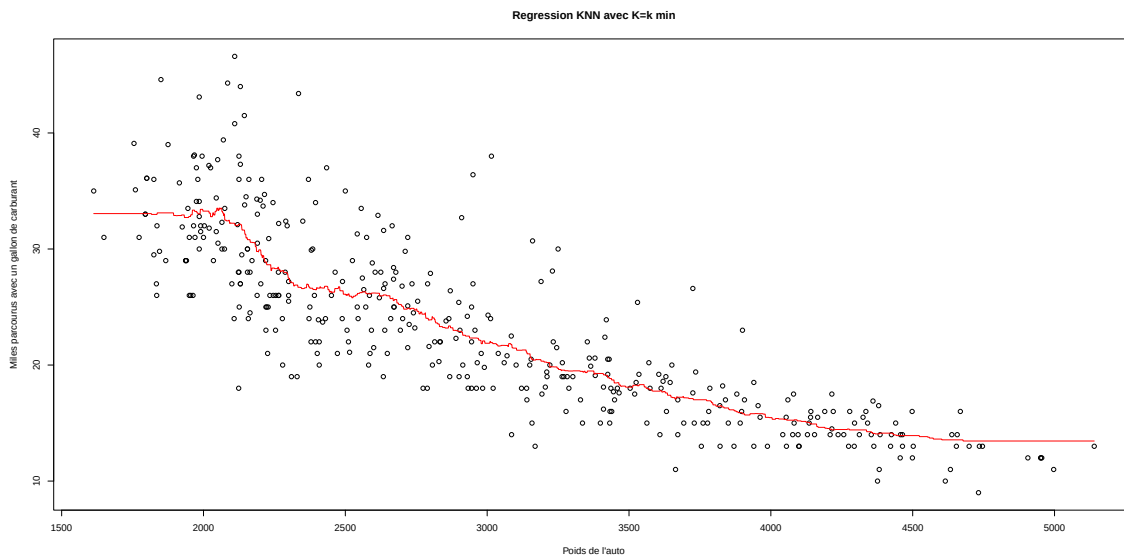
Et on obtient



La méthode de validation croisée avec le taux d'erreur test estimé en enlevant une observation à la fois donne $k_{\min} = 45$. Avec cette valeur, on a:

```
> pred_k.min <- knn.reg(train=X,test=axe.x,y=Y,k=k.min)
> plot(X,Y,type="p",xlab="Poids de l'auto",
+      ylab="Miles parcourus avec un gallon de carburant")
> lines(axe.x$lstat,pred_k.min$pred,col="red")
> title("Regression KNN avec K=k min")
```

Et on obtient



Supposons maintenant qu'on a deux variables explicatives. Les données sont donc $\{(Y_i, X_{i1}, X_{i2}), i = 1, \dots, n\}$ et le modèle est

$$Y_i = f(X_{i1}, X_{i2}) + \epsilon_i;$$

avec $E[\epsilon_i | X_{i1}, X_{i2}] = 0$. L'estimateur des k plus proches voisins de f est quasiment identique au cas avec une seule variable explicative. La seule différence est la façon de définir les distances de proximité $d_i(x)$. Dans cette situation, $d_i(x)$ est la distance entre $x = (x_1, x_2)^\top$ et $(X_{i1}, X_{i2})^\top$.

La première idée est de poser

$$\begin{aligned} d_i(x) &= (X_{i1} - x_1)^2 + (X_{i2} - x_2)^2 \\ &= \begin{pmatrix} X_{i1} - x_1 & X_{i2} - x_2 \end{pmatrix} \times \begin{pmatrix} X_{i1} - x_1 \\ X_{i2} - x_2 \end{pmatrix} \\ &= \begin{pmatrix} X_{i1} - x_1 \\ X_{i2} - x_2 \end{pmatrix}^\top \times \begin{pmatrix} X_{i1} - x_1 \\ X_{i2} - x_2 \end{pmatrix} \end{aligned}$$

Cependant, c'est possible que les ordres de grandeur de X_{i1} et X_{i2} soient différents. C'est le cas, par exemple, si X_{i1} et X_{i2} sont le poids (en kilogrammes) et la cylindrée (en pouces³) de la voiture i . En effet, X_{i1} varie de 1500 à 5000 alors que X_{i2} varie de 60 à 500. Dans cette situation, X_{i1} aura plus d'impact sur $d_i(x)$ que X_{i2} . Une façon simple de surmonter cette

difficulté est de pondérer la somme de carrés par l'inverse de la matrice de variance-covariance des variables explicatives. On obtient

$$d_i(x) = \begin{pmatrix} X_{i1} - x_1 \\ X_{i2} - x_2 \end{pmatrix}^\top \times V^{-1} \times \begin{pmatrix} X_{i1} - x_1 \\ X_{i2} - x_2 \end{pmatrix},$$

où

$$V = \begin{pmatrix} s_1^2 & s_{12} \\ s_{12} & s_2^2 \end{pmatrix},$$

$$s_1^2 = \frac{1}{n-1} \sum_{i=1}^n (X_{i1} - \bar{X}_1)^2$$

$$s_2^2 = \frac{1}{n-1} \sum_{i=1}^n (X_{i2} - \bar{X}_2)^2$$

$$s_{12} = \frac{1}{n-1} \sum_{i=1}^n (X_{i1} - \bar{X}_1)(X_{i2} - \bar{X}_2).$$

C'est la distance de Mahalanobis. On peut facilement généraliser cette distance à p variables explicatives continues.

2 Estimateur de noyau

Reconsidérons le cas où on a une seule variable explicative, $p = 1$.

L'estimateur des k plus proches voisins s'écrit

$$\begin{aligned} \hat{f}(x) &= \frac{1}{k} \sum_{i \in N_k(x)} Y_i \\ &= \frac{\sum_{i=1}^n 1_{\{i \in N_k(x)\}} Y_i}{\sum_{i=1}^n 1_{\{i \in N_k(x)\}}} \end{aligned}$$

On peut définir la notion de voisinage entre x et X_i autrement. Par exemple, on peut dire que x et X_i sont voisins si $X_i \in [x-h, x+h]$, qu'on peut encore écrire $|X_i - x| \leq h$ où $h > 0$ est un paramètre de voisinage qui joue le même rôle que k .

Ainsi, $\hat{f}(x)$ devient

$$\begin{aligned}\hat{f}(x) &= \frac{\sum_{i=1}^n 1_{\{|X_i-x|\leq h\}} Y_i}{\sum_{i=1}^n 1_{\{|X_i-x|\leq h\}}} \\ &= \frac{\sum_{i=1}^n K\left(\frac{X_i-x}{h}\right) Y_i}{\sum_{i=1}^n K\left(\frac{X_i-x}{h}\right)} \\ &= \sum_{i=1}^n w_i(x) Y_i\end{aligned}$$

où $K(u) = 1_{\{|u|<1\}}$ est appelé noyau et

$$w_i(x) = \frac{K\left(\frac{X_i-x}{h}\right)}{\sum_{j=1}^n K\left(\frac{X_j-x}{h}\right)}, \quad i = 1 \dots, n$$

sont des poids positifs qui vérifient $w_1(x) + w_2(x) + \dots + w_n(x) = 1$. Ces poids ne peuvent prendre que les valeurs 0 ou $1/A$ où $A = \sum_{i=1}^n 1_{\{|X_i-x|\leq h\}}$ est le nombre d'observations voisins de x .

Ainsi, la contribution de l'observation (X_i, Y_i) à $\hat{f}(x)$ passe subitement de $1/A$ à 0 si x sort du voisinage de X_i . Ainsi, $\hat{f}$ est une fonction discontinue de x . L'idée de l'estimateur à noyau, aussi appelé estimateur de Nadaraya-Watson est d'avoir des poids qui diminuent d'une façon continue au fur et à mesure que x s'éloigne de X_i .

À cet effet, on peut généraliser la définition du noyau K . On peut par exemple considérer le noyau Gaussien $K(u) = e^{-u^2/2}$. On obtient ainsi des poids continus et un estimateur $\hat{f}_{NW}$ continu.

Avec l'exemple considéré dans la section précédente, on calcule $\hat{f}_{NW}$ en utilisant différentes valeurs de h .

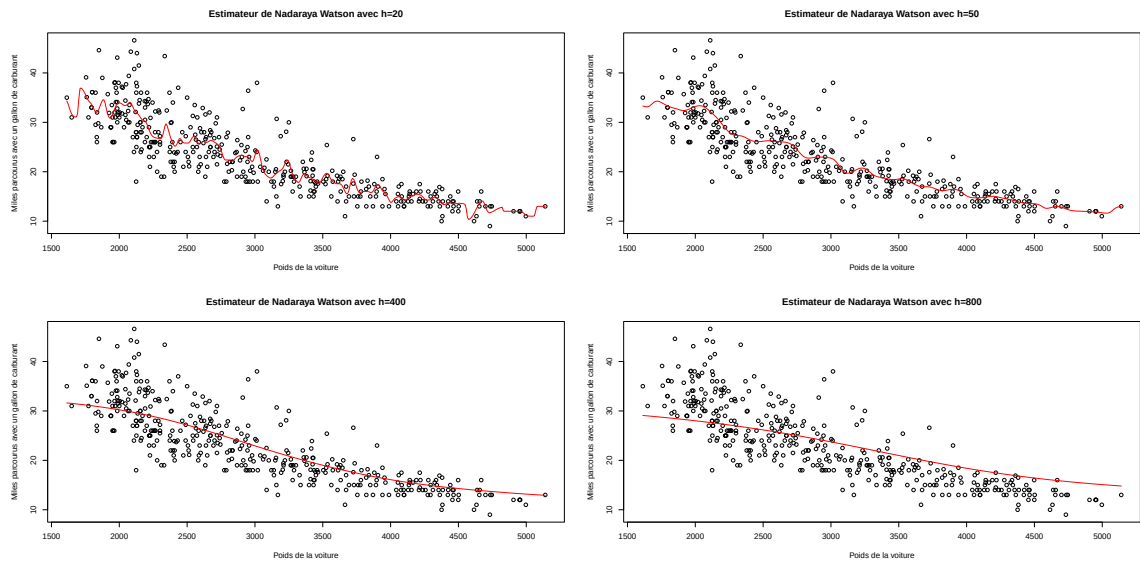
```
> library("ISLR")#### Installer la librairie ISLR
>
> data(Auto)
> X <- Auto$weight
> Y <- Auto$mpg
>
> if (!require("KernSmooth")) install.packages("KernSmooth")
> library(KernSmooth)
>
> NW20 <- locpoly(x=X, y=Y, degree=0, bandwidth=20)
```

```

> NW50 <- locpoly(x=X, y=Y, degree=0, bandwidth=50)
> NW400 <- locpoly(x=X, y=Y, degree=0, bandwidth=400)
> NW800 <- locpoly(x=X, y=Y, degree=0, bandwidth=800)
>
>
> par(mfrow = c(2,2))
>
> plot(X,Y,xlab="Poids de la voiture",
+       ylab="Miles parcourus avec un gallon de carburant")
> lines(NW20$x,NW20$y,col="red")
> title("Estimateur de Nadaraya Watson avec h=20")
>
> plot(X,Y,xlab="Poids de la voiture",
+       ylab="Miles parcourus avec un gallon de carburant")
> lines(NW50$x,NW50$y,col="red")
> title("Estimateur de Nadaraya Watson avec h=50")
>
> plot(X,Y,xlab="Poids de la voiture",
+       ylab="Miles parcourus avec un gallon de carburant")
> lines(NW400$x,NW400$y,col="red")
> title("Estimateur de Nadaraya Watson avec h=400")
>
> plot(X,Y,xlab="Poids de la voiture",
+       ylab="Miles parcourus avec un gallon de carburant")
> lines(NW800$x,NW800$y,col="red")
> title("Estimateur de Nadaraya Watson avec h=800")

```

Et on obtient



Ces graphiques nous montrent qu'on doit choisir le paramètre h . Ceci est fait avec la méthode de validation croisée. Si on exclut une seule observation à la fois, on montre que le critère de validation croisée s'écrit

$$cv.n = \frac{1}{n} \sum_{i=1}^n \left(\frac{Y_i - \hat{f}(X_i)}{1 - w_i(X_i)} \right)^2$$

Si on revient à l'exemple des autos, on a :

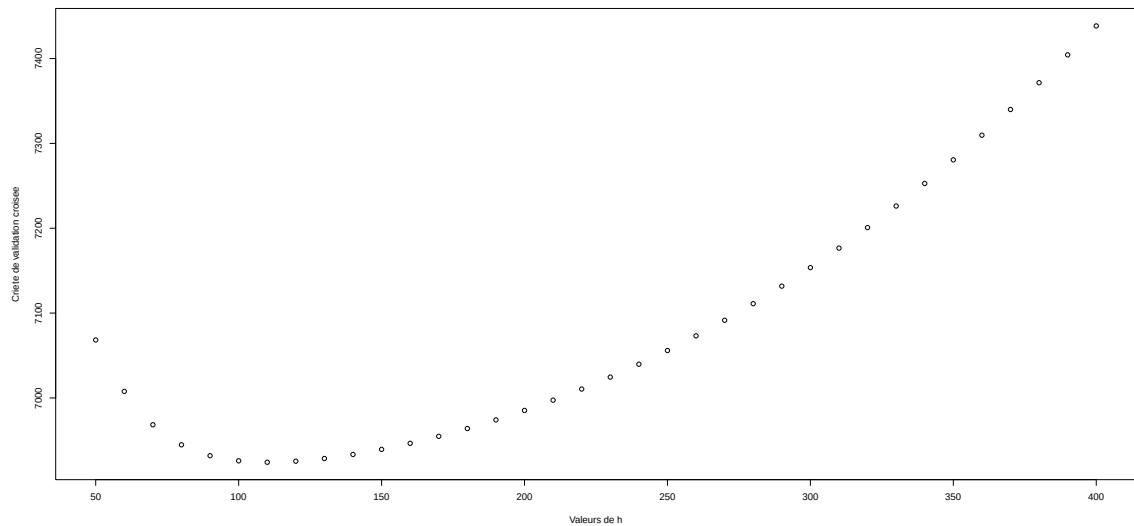
```
> #####
> ## Selection de h ##
> #####
>
> nw <- function(X,Y,h,K)
+ {
+   n <- length(X)
+   res <- NULL
+   for (i in 1:n)
+   {
+     W <- K((X-X[i])/h)
+     res <- c(res,sum(W*Y)/sum(W))
+   }
+   res
+ }
```

```

>
> CV.NW <- function(X,Y,h,K=dnorm)
+ {
+   A <- Y - nw(X=X,Y=Y,h=h,K=K)
+   B <- (1 - K(0) / colSums(K(outer(X,X,"-") / h)))
+   sum((A/B)**2)
+ }
>
>
> h.valeurs <- seq(50,400,10)
> res.cv <- NULL
>
> for(h in h.valeurs)
+ {
+   res.cv <- c(res.cv,CV.NW(X,Y,h))
+ }
>
> plot(h.valeurs,res.cv,xlab="Valeurs de h",ylab="Crite de validation croisee")
>
> h.min <- h.valeurs[which.min(res.cv)]; h.min
[1] 110

```

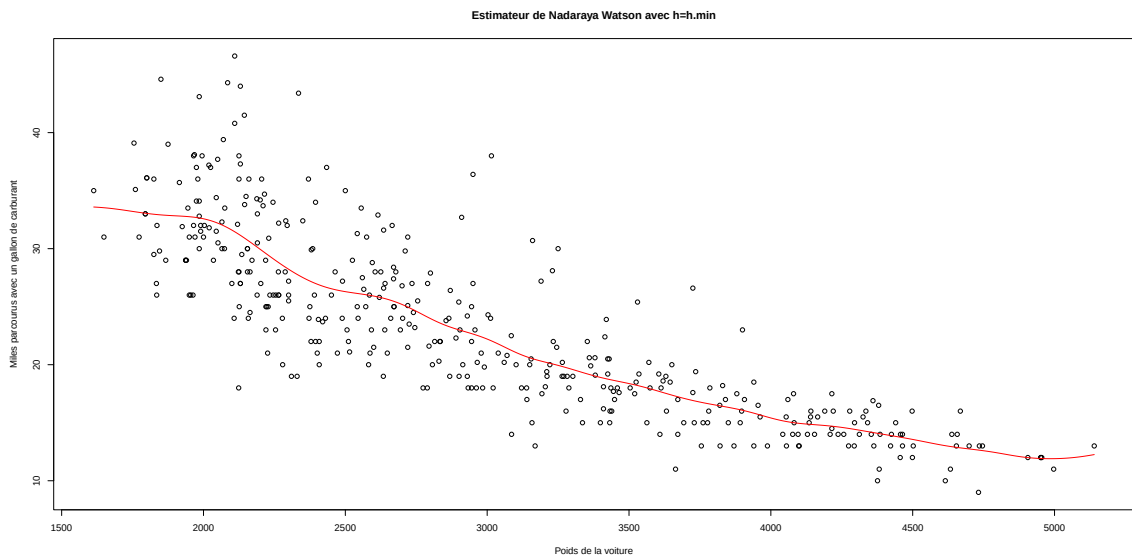
Et on obtient



Ainsi, le h qui minimise le critère de validation croisée est égal à 110. avec cette valeur, on a :

```
> NW.h.min <- locpoly(x=X, y=Y, degree=0, bandwidth=h.min)
> plot(X,Y,xlab="Poids de la voiture",
+      ylab="Miles parcourus avec un gallon de carburant")
> lines(NW.h.min$x,NW.h.min$y,col="red")
> title("Estimateur de Nadaraya Watson avec h=h.min")
```

Et on obtient



3 Régression logistique et classification

Dans les exemple précédents, on a considéré seulement le cas où la variable réponse est continue.

Considérons maintenant le jeu de données `Default` de la library `ISLR` du logiciel R. Ce jeu de données contient 10000 observations $\{(Y_i, X_i), i = 1, \dots, n\}$. La variable réponse Y_i est binaire. Elle vaut 1 si le sujet i a fait défaut et n'a pas payé sa carte de crédit et zéro sinon. La variable explicative est continue. C'est le solde à payer sur la carte de crédit.

Le modèle dans ce contexte est $g(x) = P(Y = 1|X = x)$. On estime g par

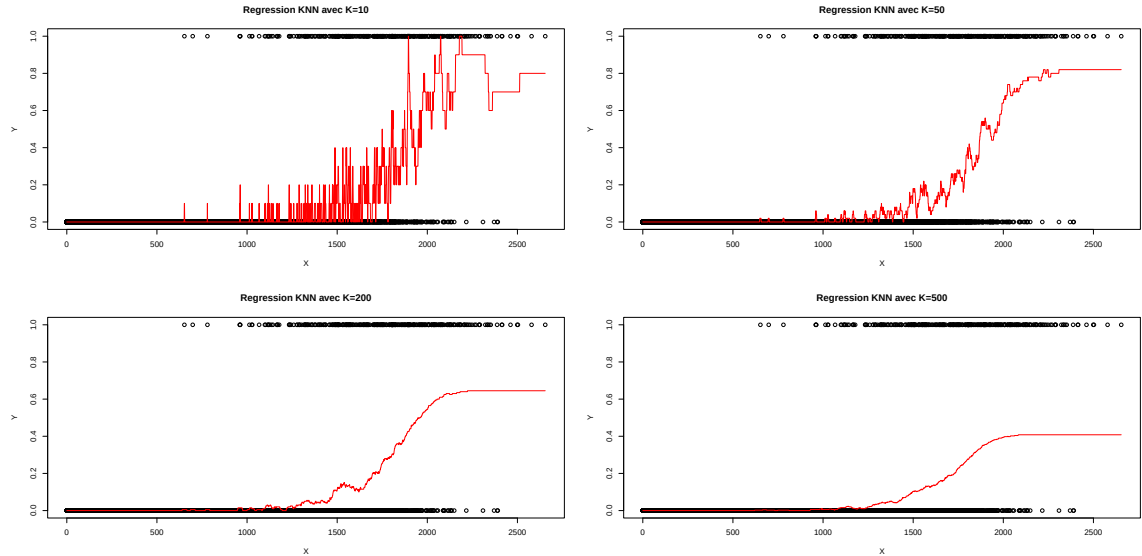
l'estimateur des k plus proches voisins

$$\hat{g}(x) = \frac{1}{k} \sum_{i \in N_k(x)} Y_i.$$

Il est facile de voir que $\hat{g}(x) \in [0, 1]$.

```
> if (!require("FNN")) install.packages("FNN")
Le chargement a nécessité le package : FNN
Message d'avis :
le package 'FNN' a été compilé avec la version R 4.4.2
> library(FNN)
>
> axe.x <- data.frame(lstat=min(X):max(X))
>
> pred_10 <- knn.reg(train=X,test=axe.x,y=Y,k=10)
> pred_50 <- knn.reg(train=X,test=axe.x,y=Y,k=50)
> pred_200 <- knn.reg(train=X,test=axe.x,y=Y,k=200)
> pred_500 <- knn.reg(train=X,test=axe.x,y=Y,k=500)
>
> par(mfrow = c(2,2))
>
> plot(X,Y,type="p")
> lines(axe.x$lstat,pred_10$pred,col="red")
> title("Regression KNN avec K=10")
>
> plot(X,Y,type="p")
> lines(axe.x$lstat,pred_50$pred,col="red")
> title("Regression KNN avec K=50")
>
> plot(X,Y,type="p")
> lines(axe.x$lstat,pred_200$pred,col="red")
> title("Regression KNN avec K=200")
>
> plot(X,Y,type="p")
> lines(axe.x$lstat,pred_500$pred,col="red")
> title("Regression KNN avec K=500")
```

Et on obtient



Ces graphiques nous montrent encore une fois l'importance de bien choisir le paramètre k . Dans la pratique, on choisit k avec la méthode de validation croisée. On peut utiliser le taux d'erreur test comme critère pour la validation croisée. Si on considère la version de la validation croisée qui exclut une observation à la fois, on pose

$$\hat{Y}_i = \begin{cases} 1 & \text{si } \hat{f}_{-i}(X_i) > u, \\ 0 & \text{sinon,} \end{cases}$$

où $u \in [0, 1]$ est un entier fixé en avance. Ici, $\hat{f}_{-i}(X_i)$ est calculé en utilisant toutes les observations, sauf la $i^{\text{ème}}$. Le taux d'erreur test est donc estimé par

$$\frac{1}{n} \sum_{i=1}^n 1_{\{y_i \neq \hat{y}_i\}}$$

Dans la pratique, on pose $u = 1/2$ et on sélectionne l'entier k qui minimise le taux d'erreur test.

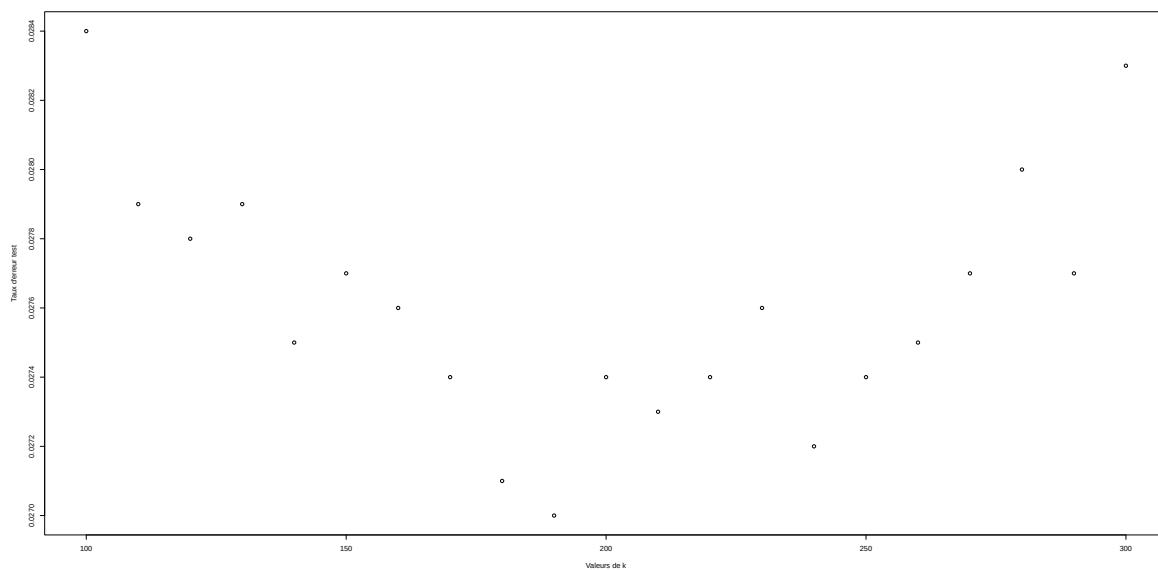
```
> ### Methode 1 : Taux d'erreur test
>
> valeurs.k <- seq(100,300,10)
> res.cv.1 <- NULL
> u <- 0.5
```

```

>
> for (k in valeurs.k)
+ {
+   pred <- knn.reg(train=X,y=Y,k=k)
+   r <- pred$residuals
+   p.hat <- Y - r
+   Y.hat <- as.numeric(p.hat>u)
+   res.cv.1 <- c(res.cv.1,mean(Y!=Y.hat))
+ }
>
> plot(valeurs.k,res.cv.1,xlab="Valeurs de k",
+       ylab="Taux d'erreur test")
>
> k.min <- valeurs.k[which.min(res.cv.1)]; k.min
[1] 190

```

Et on obtient



Le taux d'erreur test est minimal pour $k = 190$. On illustre le graphique obtenu avec $k = 190$.

```

> pred_k.min <- knn.reg(train=X,test=axe.x,y=Y,k=k.min)

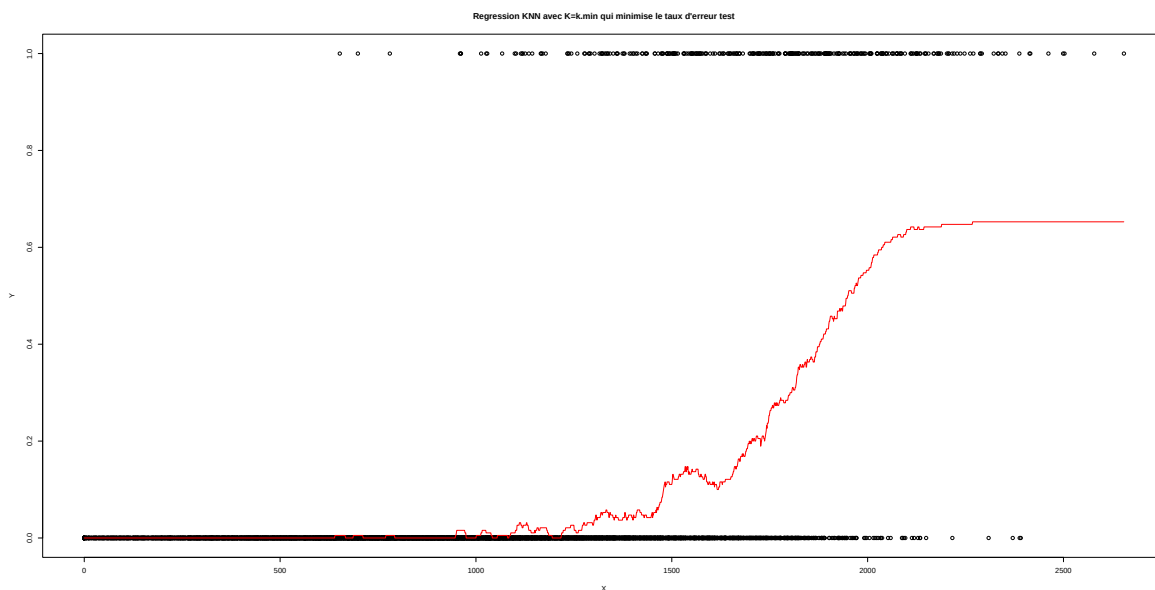
```

```

> plot(X,Y,type="p")
> lines(axe.x$lstat,pred_k.min$pred,col="red")
> title("Regression KNN avec K=k.min qui minimise le taux d'erreur test")

```

Et on obtient



La méthode décrite ci-haut pour sélectionner le paramètre k dépend de u . Par exemple, avec $u = 1/4$, on obtient $k = 290$ et avec $u = 3/4$, on obtient $k = 50$. Le choix de u est arbitraire, même si $u = 1/2$ paraît un choix naturel. Pour remédier à cette problématique, on peut considérer l'aire sous la courbe ROC comme critère pour la validation croisée. On choisira alors l'entier k qui maximisera l'aire sous la courbe.

```

> library("pROC")#### Installer la librairie pROC
>
> valeurs.k <- seq(100,300,10)
> res.cv.2 <- NULL
> for (k in valeurs.k)
+ {
+   pred <- knn.reg(train=X,y=Y,k=k)
+   r <- pred$residuals
+   p.hat <- Y - r
+ }

```

```

+ courbe.roc <- roc(response=Y,predictor=p.hat)
+ res.cv.2 <- c(res.cv.2,courbe.roc$auc)
+ }
> plot(valeurs.k,res.cv.2,xlab="Valeurs de k",
+      ylab="Surface sous la courbe")
>
> k.max <- valeurs.k[which.max(res.cv.2)]
> k.max
[1] 200

```

On sélectionne $k = 200$, ce qui est proche du k sélectionné par le taux d'erreur test avec $u = 1/2$.